GeoCool — Getting Started From Zero (College PC, i9)

Status: nothing built yet. This is the actual first-actions document — follow it in order. **Note:** GEE access will be provided by your guide, so skip any GEE account-creation steps until they hand that over — that's flagged clearly below wherever it applies.

Before you touch code: 3 things to confirm as a team (10 minutes)

Who's on this PC, and how do the other two access it? If this is a shared college lab machine, decide now: is everyone coding directly on it in turns, or is this PC just for the heavy GEE/data-processing work while Person B and C develop backend/frontend on their own laptops? This matters because it changes whether you need Docker/DB running locally on this machine specifically, or just Python.
Confirm the exact study area. Nagpur city core + Samruddhi Mahamarg interchange — get the actual bounding box coordinates locked before writing any script (see Step 2 below). Don't start downloading data with a vague "somewhere around Nagpur" — this wastes time later.
Create the GitHub repo now, even before installing anything else. One person creates it, invites the other two as collaborators. Empty repo is fine — this is so nobody's local setup work gets lost or duplicated.

Step 1 — Install software on the college PC (do this first, ~30-45 min)

Run these one at a time, verify each works before moving to the next.

1a. Git

Download from git-scm.com, install with defaults. Verify:

```bash
git --version
```

1b. Python 3.10 or 3.11 — NOT 3.12+

This matters: GeoPandas and Rasterio (which you'll need soon) frequently break on the newest Python. Download 3.11 specifically from python.org.

```
bash
python --version   # should show 3.11.x
```

1c. Node.js (LTS version)

Needed later for the frontend, but install now so it's done. Download from nodejs.org.

```
bash
node --version
npm --version
```

1d. Java JDK 17 or 21

Needed for Spring Boot later. Download from Adoptium (Eclipse Temurin) or Oracle.

```
bash
java --version
```

Set the `JAVA_HOME` environment variable after install (search "environment variables" in Windows settings, add a new System variable pointing to your JDK install folder).

1e. Docker Desktop

For running MySQL without a messy local install. Download from docker.com, install, restart if prompted, then verify:

```
```

```bash
docker --version
```

1f. VS Code

General-purpose editor for Python and React work. Download from code.visualstudio.com.

1g. IntelliJ IDEA Community Edition (only if this PC is also doing backend work)

Skip this if the Spring Boot backend is being built on a different laptop.

Checkpoint: all six `--version` commands above return a version number with no errors. Take a screenshot of this for your first mentor check-in — "Week 1 done" evidence.

Step 2 — Set up the Python environment for GIS work (15 min)

```bash
cd path\to\your\project\folder
python -m venv venv
venv\Scripts\activate       # on Windows
```

You should now see `(venv)` at the start of your terminal prompt — that confirms it's active.

Install the geospatial libraries:

```bash
pip install osmnx geopandas networkx matplotlib rasterio
```

Note on this step: these installs can be slow/finicky on Windows specifically (GeoPandas has several dependency layers). If you hit installation errors, don't burn hours debugging — the fallback is installing Anaconda/Miniconda and running:

```bash
conda install -c conda-forge osmnx geopandas rasterio networkx
```

Conda handles the tricky binary dependencies (GDAL etc.) much more reliably than plain pip on Windows.

Checkpoint: run `python -c "import osmnx, geopandas, networkx; print('all good')"` — if it prints without errors, you're set.

Step 3 — Lock your exact study area coordinates (10 min)

Before downloading anything, pin down the exact area:

Go to **bboxfinder.com**.
Draw a rectangle covering Nagpur's city core, extended to include the Samruddhi Mahamarg interchange.
Copy the bounding box (Min Longitude, Min Latitude, Max Longitude, Max Latitude) — write it in `docs/study-area.md` in your repo.
Also decide the center-point + radius alternative: Nagpur center is roughly `(21.1458, 79.0882)`. A **15km radius** from that point is a reasonable starting size that includes both the city core and reaches toward Samruddhi Mahamarg — confirm this actually captures what you want by checking it visually on bboxfinder or Google Maps first, don't just trust the number blindly.

Deliverable: one file, `docs/study-area.md`, with the final bounding box AND the center-point/radius alternative, both written down — every script from here on references this file's numbers, not numbers retyped from memory.

Step 4 — Download the road network (this is your first real deliverable)

Create `download_osm_nagpur.py`:

```python
```

```
import osmnx as ox

ox.settings.log_console = True
ox.settings.use_cache = True

print("Connecting to OpenStreetMap Overpass API...")

# Use the coordinates you locked in Step 3
nagpur_coords = (21.1458, 79.0882)
radius_meters = 15000  # 15 km — adjust if Step 3 decided differently

print(f"Downloading street network within {radius_meters}m of {nagpur_coords}...")
G = ox.graph_from_point(nagpur_coords, dist=radius_meters, network_type='all')

print("Projecting graph to metric CRS...")
G_projected = ox.project_graph(G)

nodes, edges = ox.graph_to_gdfs(G_projected)

filepath = "data/nagpur_geocool_network.gpkg"
print(f"Saving data to {filepath}...")
edges.to_file(filepath, layer='edges', driver='GPKG')
nodes.to_file(filepath, layer='nodes', driver='GPKG')

print("Success! OSM data downloaded and saved.")
```

Run it:

```
bash
python download_osm_nagpur.py
```

This will take a minute or two for a 15km radius — your i9 will handle the graph projection step fast, the slow part is just the network download itself over the Overpass API (not CPU-bound).

What this gives you: a file, `nagpur_geocool_network.gpkg`, containing your entire road network as a proper mathematical graph — every road segment already has length, road type (residential/motorway/etc.), and geometry as attributes. This is the actual foundation everything else (AHP scoring, routing) gets built on top of.

Checkpoint: the `.gpkg` file exists and is a reasonable size (tens of MB, not KB — if it's suspiciously small, the download likely failed silently; check the console output for errors).

Step 5 — Satellite data (do NOT start this until your guide provides GEE access)

This is where you stop for now. The next data layers — heat (LST), vegetation (NDVI), elevation, population — all come through Google Earth Engine, and per your guide's plan, the account/project access will be handed to you rather than each of you registering separately.

What to prepare while waiting:

Install the libraries now, so there's no delay once access arrives:

```bash
pip install earthengine-api geemap
```

Read through (don't run yet) what the extraction script will need: a Google Cloud Project ID (your guide will give you this), the same bounding box/coordinates from Step 3, and a date range for "peak summer" imagery (April–May is the standard choice for Nagpur's heat patterns).
Once your guide shares access, the actual authentication step is just running `earthengine authenticate` once per machine, logging in with whichever account/project they specify.

Don't build placeholder/fake GEE calls in the meantime — wait for real access rather than writing code against assumptions about what the project ID or permissions will look like; it's faster to write it once correctly than to write it twice.

What "done" looks like for this first work session

By the end of this session, on this college PC, you should have:

- [ ] All 6 tools installed and verified (Step 1)
- [ ] Python venv created with GIS libraries installed (Step 2)
- [ ] `docs/study-area.md` with your locked bounding box (Step 3)
- [ ] `data/nagpur_geocool_network.gpkg` — your actual road network graph (Step 4)
- [ ] `earthengine-api` and `geemap` installed and ready, waiting on your guide for access (Step 5)
- [ ] GitHub repo created, this work pushed to it

That's genuinely a full, honest Week 1 (and a head start into Week 2) — the road network is the one piece you can complete entirely independently of your guide's GEE access, so it's the right thing to prioritize doing first while you wait for that.